AjaxAdapter + AjaxProxy

Type-safe, boilerplate-free GlideAjax. Write a typed method on the server, call it like a promise on the client.

Write a typed, unit-testable method on the server. Call it from the browser like a promise. No GlideAjax boilerplate, no string-typed parameters, no guessing what went wrong.

`AjaxAdapter` (server) turns a plain private method into a client-callable endpoint. `AjaxProxy` (client) calls it and hands you back a real Promise with typed errors.

Quick start

This is all most endpoints ever need. Everything after it is reference. Read it when you want it.

1. Install once

Fastest path: import `dist/ajax-adapter-v1.1.0.update-set.xml` (Retrieved Update Sets → Import from XML → Preview → Commit), then run the Fix Script `AjaxAdapter - link Include to themes`. That lands both core records plus Service Portal auto-load and guardrails. See Install for the details and the by-hand alternative.

To do it by hand, create two platform records:

- `AjaxAdapter`: a Script Include (**Client callable: off**). Paste `ajax-adapter.script-include.js`.
- `AjaxProxy`: a UI Script (**Global: on**, **UI Type: Desktop** — not "All"). Paste `ajax-proxy.ui-script.js`. For Service Portal loading, see Install.

2. Write an endpoint (server)

A Script Include that extends `AbstractAjaxProcessor`. Pair each public method with a private one, so the public method is one line.

```
var UserLookupAjax = Class.create();
UserLookupAjax.prototype = Object.extendsObject(global.AbstractAjaxProcessor, {

  // Public: the GlideAjax boundary. Maps { userId } from the client to the private args
  // and enforces the contract: a missing/mistyped userId rejects as badRequest.
  getUserSummary: AjaxAdapter.expose('_getUserSummary', [
    { name: 'userId', type: 'string', required: true },
  ]),

  // Private: typed in, typed out, unit-testable. No getParameter, no JSON, no try/catch, no guards.
  _getUserSummary: function(userId) {
    var gr = new GlideRecord('sys_user');
```

```
    if (!gr.get(userId)) {
      throw AjaxAdapter.fail('No user matches that id'); // expected failure, safe to show
    }
    return { name: String(gr.getValue('name')), email: String(gr.getValue('email')) };
  },

  type: 'UserLookupAjax',
});
```

3. Call it (client)

```
AjaxProxy.call('UserLookupAjax', 'getUserSummary', { userId: g_form.getUniqueValue() })
  .then(summary => g_form.addInfoMessage(summary.name))
  .catch(error => g_form.addErrorMessage(error.message));
```

That's the whole loop.

Why use it

- **No boilerplate.** No `getParameter`, `JSON.parse`, `JSON.stringify`, or per-method `try/catch`. The public method is a single line.
- **Types survive the wire.** `42` arrives as `42`, not `"42"`. `null` stays `null`, `undefined` stays `undefined`. Raw GlideAjax stringifies everything.
- **Dates survive too.** A JS `Date` parameter arrives server-side as a real `GlideDateTime`; a returned `GlideDateTime` resolves client-side as a `Date`. Always the UTC instant, never a timezone-shifted string.
- **Declared parameter contracts.** `{ name, type, required, default }` at the boundary. Violations reject as `badRequest` with every problem listed at once, and your private method never runs — so it needs no guard clauses.
- **Unit-testable logic.** Private methods take typed args and return typed values. Call `new UserLookupAjax()._getUserSummary(id)` directly in ATF or any harness. No transport to mock.
- **Real promises.** `.then` / `.catch` / `.finally`, `async/await`, or callbacks, your pick, same method.
- **Typed errors.** Branch on `error.kind`, never `indexOf` a message string.
- **Safe by default.** A server bug is logged with a correlation id and anonymized to the client. No stack, table, or sys_id leaks. The console links straight to the log row.
- **Collision-proof params.** A parameter named `name`, `order`, or `constructor` can't clobber GlideAjax internals.
- **Resilient.** Every call has a timeout instead of hanging forever, with opt-in retry for transient failures.

Before and after

Calling from the client

```
// Raw GlideAjax: string params, manual parse,
// no idea if the answer is data or an error.
var ga = new GlideAjax('UserLookupAjax');
ga.addParam('sysparm_name', 'getUserSummary');
ga.addParam('sysparm_userId', userId);
ga.getXMLAnswer(function(answer) {
  var summary = JSON.parse(answer);
  render(summary);
});
```

A promise with typed errors

```
AjaxProxy.call('UserLookupAjax', 'getUserSumma
ry', { userId })
  .then(render)
  .catch(error => showError(error.message));
```

Writing the server method

```
getUserSummary: function() {
  try {
    var userId = new String(this.getParameter(
'sysparm_userId'));
    var gr = new GlideRecord('sys_user');
    if (!gr.get(userId)) {
      return JSON.stringify({ error: 'not foun
d' });
    }
    return JSON.stringify({ name: '' + gr.name
 });
  } catch (e) {
    gs.error(e);
    return JSON.stringify({ error: 'server err
or' });
  }
},
```

One line + pure logic

```
getUserSummary: AjaxAdapter.expose('_getUserSu
mmary', ['userId']),

_getUserSummary: function(userId) {
  var gr = new GlideRecord('sys_user');
  if (!gr.get(userId)) {
    throw AjaxAdapter.fail('No user matches th
at id');
  }
  return { name: String(gr.getValue('name')) }
;
},
```

Handling errors

```
ga.getXMLAnswer(function(answer) {
  if (!answer) { /* ACL? typo? timeout? */ }
  var data = JSON.parse(answer);
  if (data.error &&
      data.error.indexOf('not found') > -1) {
    // brittle string matching
  }
});
```

Branch on a field

```
.catch(error => {
  if (error.kind === AjaxProxy.ErrorKind.BUSIN
ESS) {
    showUser(error.message);          // safe,
 authored message
  } else {
    logBug(error.reference);          // serve
r bug, ref links to log row
  }
});
```

Passing a number

```
ga.addParam('sysparm_count', count);
// server receives "42" (a string).
// getValue math and comparisons break.
```

Type preserved

```
AjaxProxy.call('X', 'm', { count });
// server receives 42 (a number).
```

Features

Each of these is independent. Reach for one when you want it, none of them complicate the basic `call`.

Promise or callbacks, same method

Pass handlers instead of chaining when it reads better. `onComplete` runs on both success and failure.

```
AjaxProxy.call('UserLookupAjax', 'getUserSummary', { userId }, {
  onSuccess: render,
  onError:   showError,
  onComplete: hideSpinner,
});
```

Typed errors you branch on

Every rejection carries `kind` (see the table), plus `scriptInclude`, `method`, and, for a bug, a `reference`. No message parsing, ever.

```
.catch(error => {
  switch (error.kind) {
    case AjaxProxy.ErrorKind.BUSINESS: return showUser(error.message);
    case AjaxProxy.ErrorKind.TIMEOUT:  return retryLater();
    default:                           return showGeneric(error.reference);
  }
});
```

Timeout, plus opt-in retry

Every call times out instead of hanging forever. Retry is **off by default**. Opt in for transient failures on idempotent (read) methods.

```
AjaxProxy.call('X', 'm', params, { timeout: 15000 });                  // times out, no retry
AjaxProxy.call('X', 'read', params, { retry: true });                  // retry a timeout once
AjaxProxy.call('X', 'read', params, { retry: { attempts: 3, delay: 300 } });
```

> **Writes:** a timeout doesn't prove the server didn't run the method, so retry can double-apply a create/update. Leave retry off (the default) for non-idempotent methods.

`AjaxProxy.for()`: bind a script include once

Stop repeating the script-include name and shared options at every call site. `AjaxProxy` is a singleton (there's no `new`, it's a namespace of functions), so `for()` is the way to get a reusable, pre-configured caller. It returns a plain function.

```
const users = AjaxProxy.for('UserLookupAjax', { timeout: 15000, onError: toast });
users('getUserSummary', { userId }).then(render);
users('getActiveUserCount', {}).then(setBadge);
```

`AjaxProxy.channel()`: type-ahead without race conditions

For live search: debounce the input and drop stale responses so only the newest result ever lands, even if an older request answers later. Superseded calls are abandoned (their promise never settles), exactly like RxJS `switchMap`.

```
const search = AjaxProxy.channel('UserLookupAjax', 'searchUsers', { debounce: 250 });
input.addEventListener('input', () => search({ query: input.value }).then(render));
```

`AjaxProxy.setErrorHandler()`: one place for failures

Route every unhandled failure through your own sink: a toast, a banner, telemetry. Applies to callback-style calls that don't pass their own `onError`.

```
AjaxProxy.setErrorHandler(error => toast(error.message));
```

Console-to-log in one click

On a server bug, the default handler prints a deep link to the exact log row (`message STARTSWITH <reference>`). Nothing leaks: the link only opens for users with log access.

```
AjaxProxy.setLogTable('syslog_app_scope'); // point it at a scoped app's log
AjaxProxy.logUrl(error);                   // build the same link in a custom handler
```

Reference

Everything above in full. Read top-to-bottom or jump to what you need.

Install

Recommended — the update set. Import `dist/ajax-adapter-v1.1.0.update-set.xml` (Retrieved Update Sets → Import Update Set from XML → Preview → Commit), then run the Fix Script `AjaxAdapter - link Include to themes`. It installs both core records, wires `AjaxProxy` onto every Service Portal theme, and adds guardrails that keep the UI Script on Global + UI Type Desktop + Active and protect the records from deletion. Uninstall by setting the system property `ajaxadapter.portal.lock` to `false`, then deleting the records.

By hand — create the two core records:

Record	Type	Setting	Source
`AjaxAdapter`	Script Include	Client callable **off**	`ajax-adapter.script-include.js`
`AjaxProxy`	UI Script	**Global: on**, **UI Type: Desktop** (not "All")	`ajax-proxy.ui-script.js`

Service Portal (manual). The Global flag is a classic-UI loader; load `AjaxProxy` on a portal explicitly — either **portal-wide** by adding it as a **JS Include** (Source = UI Script → `AjaxProxy`) to your portal **Theme**,

or **per widget** by adding that JS Include to a **Dependency** on the specific widgets that need it. `AjaxAdapter` must be reachable from your endpoint's scope (**Accessible from: All application scopes** if you share it).

> **UI Type must be `Desktop`, not `All`.** The global UI-Script auto-loader only emits `Desktop`-typed scripts; `All` (or `Mobile`) removes it from that bucket, so it silently stops loading in **both** the classic UI and Service Portal — even a top-level `console.log` won't fire. Keep `Global: on` + `UI Type: Desktop`. (The update set enforces this for you.)

`AjaxProxy.call(scriptInclude, method, params, options?)`

`params` is a plain object, sent as one JSON payload. `options` is optional.

Option	Default	Meaning
`onSuccess` / `onError` / `onComplete`	none	Callback style. Passing any of them opts out of promise style. `onComplete` runs on both paths.
`timeout`	`60000`	Milliseconds before the call rejects with `timeout`.
`retry`	off	`true` to retry a `timeout` once, a number for total attempts, or `{ attempts, delay, on }`. Only for idempotent (read) methods.

Returns a `Promise`. In callback style it still returns the promise, so you can mix if you must.

Related: `AjaxProxy.for(scriptInclude, defaults?)`, `AjaxProxy.channel(scriptInclude, method, { debounce, latest, timeout, retry })`, `AjaxProxy.setDefaultTimeout(ms)`, `AjaxProxy.setErrorHandler(fn)`, `AjaxProxy.setLogTable(name)`, `AjaxProxy.logUrl(errorOrReference)`.

`error.kind`

`kind`	When	What to do
`business`	`AjaxAdapter.fail(...)` on the server	Show `error.message`, it's authored and safe.
`server`	An unexpected server bug	Show a generic message. `error.reference` deep-links to the log.
`badRequest`	Params weren't serializable, payload wasn't a valid JSON object, a parameter contract was violated, or the payload was oversized	Fix the caller. `error.message` lists every violation.
`timeout`	No answer in time	Safe to retry manually, or opt into `retry` for reads.
`empty`	Empty answer	Check `client_callable`, ACLs, the method name, or the session.
`malformed`	Answer wasn't an AjaxAdapter envelope	You're calling a non-AjaxAdapter script include.

Constants live on `AjaxProxy.ErrorKind` (`BUSINESS`, `SERVER`, `BAD_REQUEST`, `TIMEOUT`, `EMPTY`, `MALFORMED`).

Server: `expose()` and `fail()`

`AjaxAdapter.expose('_privateName', [...])` returns the public method. Each entry maps, in order, to one of the private method's arguments — a plain string for the mapping alone (an omitted parameter arrives as `undefined`, so keep optional ones last), or a contract object to have the boundary validate for you:

```
getUserSummary: AjaxAdapter.expose('_getUserSummary', [
  { name: 'userId', type: 'string', required: true },
  { name: 'options', type: 'object', default: {} },
]),
```

`type` is one of `string` / `number` / `boolean` / `object` / `array` / `date` (`date` matches a marshalled JS `Date`, which arrives as a `GlideDateTime`). `required` rejects when the key is absent; `default` fills it instead. Violations reject with kind `badRequest` — all listed in one message — and the private method never runs, so it can skip its guard clauses. Nothing is logged: a caller bug is not a server bug. A malformed contract itself (an unknown `type`) throws when the script include loads, not on the first call.

Two ways for a private method to fail:

```
// Expected failure the caller must handle: kind 'business'. Message shown, NOT logged.
throw AjaxAdapter.fail('This user has no manager assigned', { details: { userId } });
```

```
// Contract violation / bug: kind 'server'. Logged with a correlation id, anonymized to the client.
throw new Error('userId is required');
```

Returning normally sends the value as the result. Absence that's a normal branch is a value too (`return { found: false }`), and the client resolves with it. Reserve `fail()` for failures the caller must act on, and plain `throw` for "this should never happen."

Return plain JS values — coerce fields with `String(gr.getValue('name'))`, `Number(ga.getAggregate('COUNT'))` (`gr.getValue('name') + ''` coerces the same way if you want the shorthand; the examples in this repo spell out `String(...)` for clarity). Two exceptions the adapter handles for you: a `GlideDateTime` (or server `Date`) is marshalled to a client `Date` automatically, and any *other* Java object (a `GlideElement`, a `GlideRecord`) is rejected with a loud, logged server error instead of silently serializing as `{}`.

How it works

- **Wire format.** All parameters ride as one JSON object under `sysparm_payload`. Because the whole object is JSON, every value keeps its type, and a parameter named like a GlideAjax internal (`name`, `processor`) is just a key, so it can't collide. Only `sysparm_name` (the method) and `sysparm_processor` (the script include) are reserved by GlideAjax. The proxy also sends its version under `sysparm_adapter_version`; the server logs a warning when the two files' major.minor drift apart, since they share this wire format.
- **Dates.** A date-time travels as a single-key tag, `{ "$dateTime": "<ISO 8601 UTC>" }` — readable in the network tab, unambiguous, always the instant. The proxy turns a `Date` parameter into the tag and the adapter revives it as a `GlideDateTime`; a returned `GlideDateTime` (or server `Date`) makes the reverse trip and resolves as a client `Date`. Only an object whose *sole* key is the tag converts, so ordinary data can't collide. Opt out per call with `{ dates: false }`. Display formatting stays yours: to show a date the way ServiceNow formats it for the user, return `gdt.getDisplayValue()` as a plain string instead.
- **Envelope.** The server always answers with a JSON string: `{ ok: true, result }` or `{ ok: false, error: { kind, message, reference?, details? } }`. `AjaxProxy` unwraps it, you never see it.
- **Two-tier errors.** A plain `throw` is logged server-side under a `gs.generateGUID()` reference and returned as a generic `server` error (no internal detail leaves the instance). `AjaxAdapter.fail(...)` passes your message through untouched as `business` and is not logged.
- `undefined` **round-trips as** `undefined`, `null` as `null`. The server gets exactly what you passed, and vice versa.
- **Guardrails.** Returning a `GlideElement` or any other Java object fails loudly (a logged server error naming the offending key) instead of silently serializing as `{}`. Payloads over `AjaxAdapter.MAX_PAYLOAD_LENGTH` (default 1,000,000 characters, reassignable) reject as

`badRequest`.

Retry and idempotency

Retry is **off by default**. Opt in with `retry: true` (retry `timeout` once, with exponential backoff), a number of attempts, or `{ attempts, delay, on }`. Retry is transparent, same resolve/reject contract. Because a timeout doesn't prove the server skipped the work, **only enable retry for idempotent (read) methods**: retrying a write can double-apply it. Retry also up to doubles the worst-case wait before a call finally fails.

`channel()` semantics

A superseded call is *abandoned*: its promise never resolves or rejects, so only the latest call settles. Consequence: a stale call's `.finally()` / `.then()` never runs, so don't hang cleanup you always need off a channel call. `latest` (default `true`) drops out-of-order responses. `debounce` waits for quiet before firing.

Reusing the logic server-side

Extending `AbstractAjaxProcessor` only **adds** the AJAX plumbing, it doesn't stop you using the class as ordinary server code. The private methods take typed args and never touch `this.request`, so any server caller can use them directly:

```
// In a Business Rule, scheduled job, or another script include:
var summary = new UserLookupAjax()._getUserSummary(current.getUniqueValue());
```

Splitting the logic into a separate AJAX-free Service script include is an optional preference (keeping a domain layer off the AJAX base class), **not** a requirement for reuse.

Testing

Private methods are the unit under test. Call them directly, no transport:

```
var result = new UserLookupAjax()._getUserSummary('62826bf03710200044e0bfc8bcbe5df1');
```

They receive real typed arguments and return real typed values, so an ATF server test (or any harness) exercises the exact code the endpoint runs.

Calling without `AjaxProxy`

Any GlideAjax caller can hit the same endpoint. Build the payload yourself. There's no separate parameter format to learn.

```
var ga = new GlideAjax('UserLookupAjax');
ga.addParam('sysparm_name', 'getUserSummary');
ga.addParam('sysparm_payload', JSON.stringify({ userId: id }));
ga.getXMLAnswer(function(answer) { /* { ok, result } | { ok, error } */ });
```